AMS Pathway Project 1

Scenario

A team has a messy shared folder where people keep dropping files such as reports, meeting notes, invoices, images, and exports. Nothing is named consistently, old files pile up, and nobody knows what has been processed.

Tech Requirements

- Windows 10 / 11
- Visual Studio Code - GitHub Copilot set up
- Python
- Git
- Docker
- Linux/Bash

MVP

Build a **containerised Python file organiser and reporting tool** that:

- Scans an input folder
- Validates filenames against agreed rules
- Sorts files into the correct folders
- Archives older files
- Logs what happened
- Generates a summary report

Justification

- **Python**: file handling, functions, validation, logging, reports
- **Git**: version control, branches, commits, collaboration
- **Docker**: run the tool in a repeatable container
- **Linux**: directory structure, permissions, Bash scripts, running from terminal
- **Prompting / AI**: generate filename rules, test cases, README drafts, debugging help, refactoring ideas

Objective

Build **File-Flow**, a lightweight command-line file processing tool for a shared team folder!

Core User Story

“As a user, I want to drop files into a shared folder and run a simple tool that validates, organises, logs, and reports on those files so that the shared area stays tidy and usable.”

Required features

Learners must implement these:

- Scan a chosen input folder
- Detect files available for processing
- Validate filenames against a naming convention
- Classify files by type or naming rule
- Move valid files to the correct output folder
- Move invalid files to a quarantine folder
- Generate a text or CSV summary report
- Record actions to a log file
- Support repeated runs without crashing
- Save configuration in a file such as JSON

Deliverables

- Git repository
- Python source code
- README.md
- Dockerfile
- One or more Bash scripts
- Sample input files
- Config file
- Log file sample
- Generated report sample
- Short AI usage log
- Short demo presentation

3-day outline

Day 1

Morning

- Read the brief and identify the smallest useful version of FileFlow.
- Define the file naming standard.
- Decide the folder structure for:
  - input
  - processed
  - quarantine
  - archive
  - reports
  - logs
- Create the Git repository.
- Make the initial commit.
- Use AI to help draft:
  - backlog items
  - acceptance criteria
  - possible file classification rules
  - a starter project structure

Afternoon

Build the first version that can:

- read files from the input folder
- validate filenames
- move valid files to one destination
- move invalid files to quarantine
- write a simple log
- print a basic summary to the console

End-of-day checkpoint

By the end of Day 1, you should have:

- a working repo
- meaningful early commits
- a Python app that runs from the terminal
- filename validation working
- valid and invalid files being separated

Day 2

Morning

Extend the app so that it can:

- load rules from a JSON config file
- sort valid files into different folders by type
- generate a proper summary report
- count processed, quarantined, and archived files
- handle duplicate names safely
- handle missing folders without crashing
- improve exception handling and user feedback

Afternoon

Add delivery and runtime support:

- create a Dockerfile
- build the Docker image
- run the tool inside Docker
- mount host folders so files persist outside the container
- create a Bash script such as:
    - run_fileflow.sh
    - reset_demo_data.sh
    - backup_reports.sh

End-of-day checkpoint

By the end of Day 2, you should have:

- core functionality complete
- a working container build
- at least one useful shell script
- clearer repo history, ideally with branches used sensibly

Day 3

Morning

Complete and improve the solution:

- add archive behaviour for older processed files
- improve config handling
- improve log readability
- test with a larger sample dataset
- tidy menu or command-line options
- add helpful README instructions
- test local run versus Docker run

Afternoon

Prepare final evidence:

- finalise the AI usage log
- finalise the README
- add sample files and expected outputs
- prepare a short demo showing:
    - input folder before run
    - processing run
    - processed/quarantine/archive folders after run
    - generated report
    - Docker run
    - Git history overview

End-of-project checkpoint

By the end of Day 3, you should have:

- a finished demonstrable solution
- clean documentation
- evidence of Git, Docker, Linux, and AI use
- a presentation-ready capstone